

Contents

Архитектура, стоящая за проектом	2
1. Проект не только о слайдере	2
2. Почему сознательно сделать проект сложнее?	2
3. Проект как архитектурная лаборатория	3
4. Центральное философское правило: владение ответственностью	4
5. Почему Slider стал иерархией	5
6. Почему проект также сильно опирается на композицию	5
7. Почему менеджеры имеют такое значение	6
8. Идея EventManager	7
9. Почему нужны dynamic subscriptions	7
10. Почему _pipe() является ключевой архитектурной идеей	8
11. Почему pipeline требует контроля на уровне приложения	9
12. Почему FSM локальны	9
13. Почему не используется принудительный State pattern	10
14. Значение slidemove и slidechange	10
15. Почему autoscroll - больше, чем timer	11
16. Почему LOCKED отличается от OFF	12
17. Почему у AudioPlayer собственная модель состояния	12
18. Почему компоненты общаются через события	13
19. Проект использует паттерны как язык, а не как украшение	13
20. Почему важны DI и composition	14
21. Почему проект избегает god object	15
22. Почему validation - часть философии	15
23. Проект как этап развития	16
24. Что в итоге действительно достигнуто	16
25. Почему точное описание здесь - "сложно, но контролируемо"	17
26. Почему эта архитектура интересна не только в рамках этого проекта	17
27. Финальный взгляд	18

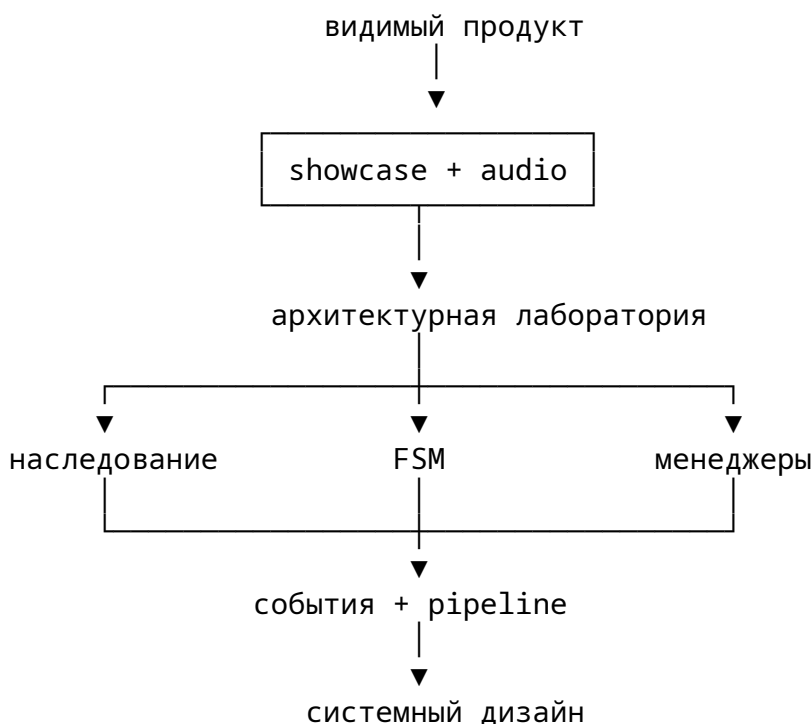
Архитектура, стоящая за проектом

Проект не только о слайдере

Снаружи проект может выглядеть почти намеренно простым: интерактивная витрина, слайдер, несколько элементов управления и аудиоплеер. Именно эта простота и делает его удобным материалом для архитектурного эксперимента.

Проект создан не потому, что слайдеру объективно требовалась необычно сложная реализация. Он создан потому, что знакомый и понятный UI может стать лабораторией для идей, которые значительно шире самого UI.

Настоящий предмет проекта - архитектура.



Продукт достаточно конкретен, чтобы с ним взаимодействовать, но достаточно мал, чтобы оставаться мысленно контролируемым. Поэтому каждое архитектурное решение можно увидеть.

1. Почему сознательно сделать проект сложнее?

Простой слайдер можно реализовать несколькими функциями и обработчиками событий. Это решило бы непосредственную UI-задачу.

Но это не та задача, которую должен был решать этот проект.

Настоящей учебной задачей было перейти от функциональной/процедурной реализации к объектно-ориентированной архитектуре и, в частности, исследовать prototype-based ООР, а не просто взять старый код и везде добавить `this`.

Это важное различие.

Старая реализация



Поэтому проект стал контролируемой средой, в которой можно было задавать вопросы:

- Где должна жить эта ответственность?
- Что должен знать компонент?
- Что следует вынести в переиспользуемую инфраструктуру?
- Что относится к наследованию, а что к композиции?
- Где состояние локально, а где должна происходить координация?
- Когда паттерн действительно полезен, а когда он лишь украшение?

Сложность здесь намеренна. Она и есть материал упражнения.

2. Проект как архитектурная лаборатория

Проект создавался как учебная и портфолио лаборатория, а не как production-приложение, которое пытается оправдать enterprise-архитектуру.

Это меняет критерий успеха.

Вопрос не в том:

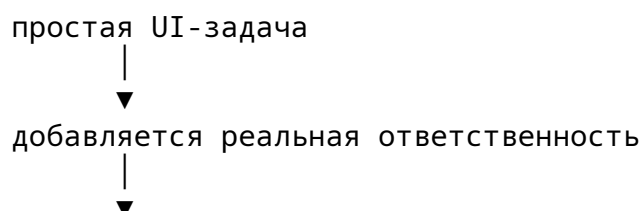
Можно ли получить тот же визуальный результат, написав меньше кода?

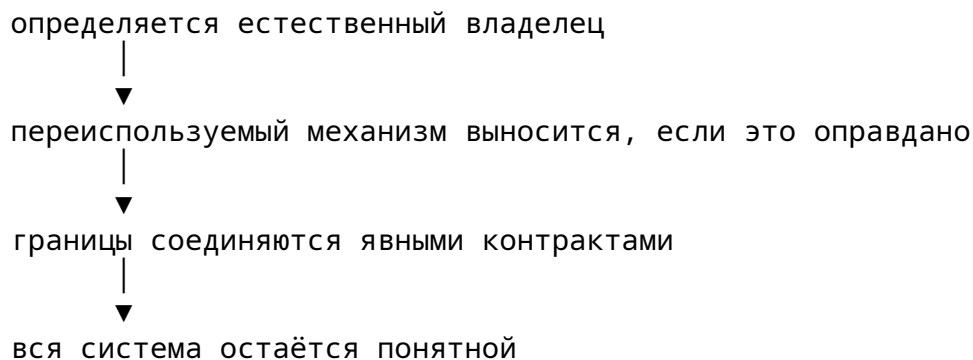
Разумеется, можно.

Гораздо интереснее другое:

Может ли намеренно более богатый дизайн оставаться понятным, внутренне непротиворечивым и контролируемым, когда одновременно работают несколько независимых интеракционных областей?

Проект исследует именно это на реальной интерактивной системе, а не на абстрактных примерах.





Последний шаг так же важен, как и первые. Архитектура полезна только до тех пор, пока разработчик способен рассуждать о результате.

3. Центральное философское правило: владение ответственностью

Проект можно понять через один вопрос:

Кто владеет этим знанием?

Ответ намеренно различается для разных типов знаний.

Slider

└─ владеет движением и состоянием взаимодействия

AudioPlayer

└─ владеет воспроизведением и состоянием аудио

Shop

└─ владеет поведением внешней ссылки

Managers

└─ владеют переиспользуемыми механизмами

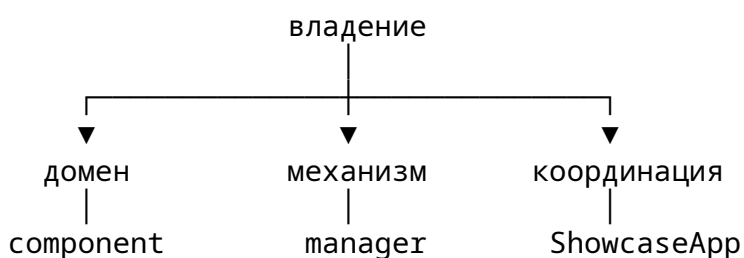
ShowcaseApp

└─ владеет координацией на уровне приложения

Это важнее любого отдельного имени класса или паттерна.

Когда владелец определён, многие проектные решения становятся проще.

Slider не должен знать, как проигрывается альбом. AudioPlayer не должен знать, как слайдер телепортируется между clone-слайдами. Менеджеру не нужно знать, что такое альбом. А ShowcaseApp должен координировать эти части, не превращаясь при этом в место, где заново реализованы все их внутренние алгоритмы.

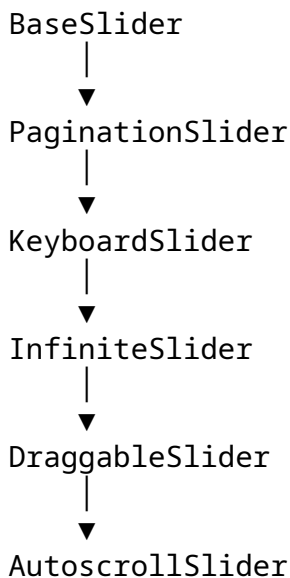


Именно это не позволяет архитектуре схлопнуться в один огромный объект.

4. Почему Slider стал иерархией

Слайдер - самое заметное проявление объектно-ориентированного мышления в проекте.

Вместо одного огромного объекта, содержащего navigation, pagination, keyboard, infinite loop, dragging и autoscroll, проект постепенно наращивает набор возможностей.



Важно не просто то, что используется inheritance. Важно, что у каждого уровня есть собственная причина существования.

Финальный объект всё ещё концептуально является одним Slider, но его возможности были приобретены последовательно.

Это делает наследование способом выразить отношение identity + extension:

тот же концептуальный объект
+
новая возможность
v
новый специализированный уровень

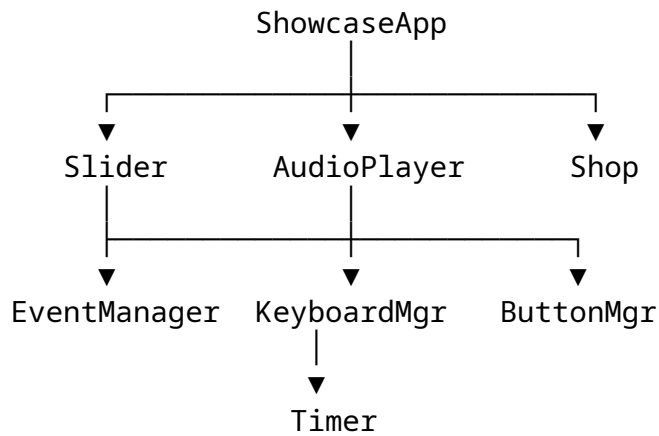
Именно поэтому проект оказался полезен для изучения prototype-based OOP. Prototype-based JavaScript делает механику наследования видимой, а не прячет её за синтаксисом class.

5. Почему проект также сильно опирается на композицию

Одного наследования было бы недостаточно.

Переиспользуемая инфраструктура не является частью Slider hierarchy. EventManager не является специальным видом Slider. Timer не является специальным видом Audio-Player.

Поэтому эти механизмы компоненты получают через composition.



В архитектуре используются оба основных OOP-механизма, но по разным причинам:

Inheritance

-> специализировать уже существующий доменный объект

Composition

-> дать компоненту переиспользуемые возможности/механизмы

Проект не пытается доказать, что один механизм всегда лучше другого. Он исследует, где каждый из них уместен.

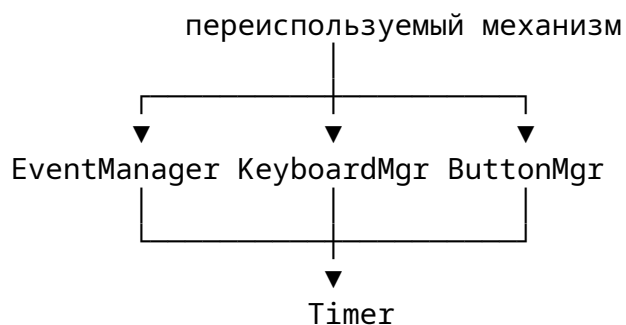
6. Почему менеджеры имеют такое значение

Менеджеры - одни из наименее заметных частей UI и одни из самых важных частей архитектуры.

Без менеджеров каждый компонент постепенно накапливал бы универсальную браузерную механику:

addEventListener()
 removeEventListener()
 сопоставление клавиш
 поиск кнопок
 валидация обработчиков
 учёт таймеров

Вместо этого проект предлагает сделать эти механизмы переиспользуемой инфраструктурой.



Архитектурный выигрыш - не только в повторном использовании кода.

Меняется сам язык компонентов.

Компонент может сказать:

"выполни next"

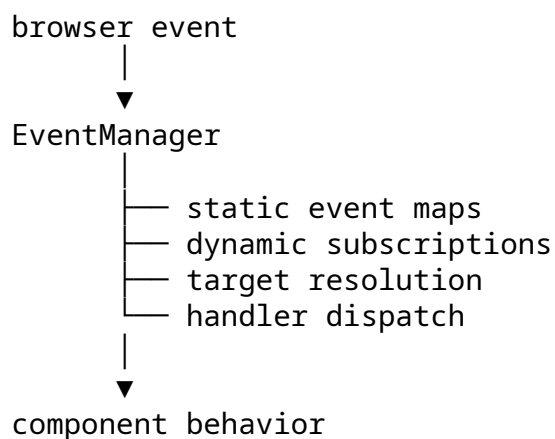
вместо:

"разбери этот KeyboardEvent, сравни клавиши, найди метод, проверь его и затем вызови"

Механизм централизован. Доменный смысл остаётся локальным.

7. Идея EventManager

EventManager особенно важен потому, что создаёт инфраструктурную границу между браузерными событиями и логикой компонентов.



Благодаря этому wiring событий становится декларативным, а не набором повторяющегося императивного setup-кода.

Prototype hierarchy делает это ещё интереснее: event maps могут принадлежать отдельным capability-слоям, а менеджер проходит по цепочке и строит итоговый набор подписок.

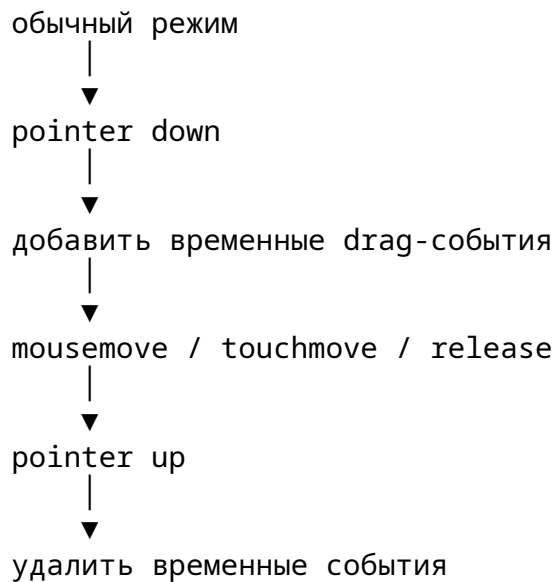
Это архитектурное выражение того же принципа, который используется в Slider hierarchy:

Пусть каждый уровень владеет той частью задачи, которая относится именно к нему.

8. Почему нужны dynamic subscriptions

Не каждое событие является постоянной частью приложения.

Drag - самый очевидный пример.



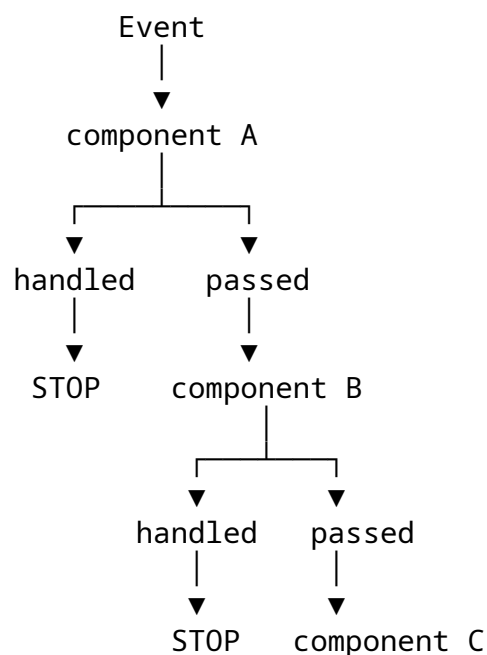
Это небольшая деталь, но у неё есть более общий архитектурный вывод: взаимодействие имеет lifecycle.

Поэтому проект рассматривает время жизни подписки как часть дизайна, а не предполагает, что все обработчики должны существовать вечно.

9. Почему `_pipe()` является ключевой архитектурной идеей

Pipeline - место, где начинают работать вместе несколько в остальном независимых идей.

Одно браузерное событие можно предложить нескольким компонентам последовательно.



Проект использует исходный Event как лёгкий continuation token. Обработчик, который поглотил ввод, может вернуть `true`; обработчик, который его не поглотил, сохраняет исходное событие, чтобы следующий участник мог попробовать его обработать.


```
true
└─ "Я это обработал."
```

```
original event
└─ "Я это не обработал. Продолжай."
```

Поэтому `_pipe()` - не просто удобный цикл. Это маленький протокол кооперации.

Он напоминает Chain of Responsibility, но без навязывания приложению textbook-иерархии handler-классов.

10. Почему pipeline требует контроля на уровне приложения

Одно и то же событие не всегда должно первым попадать одному и тому же компоненту.

В обычном режиме pipeline может отдавать приоритет Slider. В специальной ситуации с аудио он может быть развёрнут.

```
NORMAL
Slider -> AudioPlayer -> Shop
```

```
REVERSE
AudioPlayer -> Slider -> Shop
```

Это хороший пример того, почему pipeline становится архитектурным механизмом, а не просто helper-функцией.

Приложение может менять порядок, не переписывая сами компоненты.

ShowcaseApp поэтому определяет routing policy, а компоненты по-прежнему решают, что для них означает обработка события.

```
policy          -> ShowcaseApp
routing         -> pipeline
meaning        -> component
```

Это одна из самых чистых архитектурных границ проекта.

11. Почему FSM локальны

Проект содержит несколько типов состояния, но они не принадлежат одной концептуальной машине.

```
Slider
└─ IDLE / MOVING
```

```
AutoscrollSlider
└─ OFF / ON / LOCKED
```

```
AudioPlayer
└─ IDLE / THEME / ALBUM / ALBUMTHEME
```

Эти FSM взаимодействуют, но не владеют друг другом.

Slider state -> lifecycle движения
Autoscroll state -> lifecycle автоматического движения
Audio state -> контекст воспроизведения

Глобальная FSM должна была бы описывать комбинации вроде:

MOVING + LOCKED + ALBUM
IDLE + ON + THEME
...

Это быстро превращается в матрицу несвязанных обязанностей.

Локальный подход оставляет каждую модель состояния рядом с кодом, который её понимает.

12. Почему не используется принудительный State pattern

Это ещё один пример общей философии работы с паттернами.

Проект использует явные FSM, но не вводит большую иерархию State-объектов только потому, что в системе существует состояние.

реальная проблема
 v
требуется моделирование состояния
 v
локальной FSM достаточно
 v
дополнительная абстракция не нужна

State pattern стал бы полезен, если бы state-specific behavior настолько разрослось, что отдельные объекты состояния действительно сделали бы систему понятнее.

Пока локальная FSM яснее.

Принцип прост:

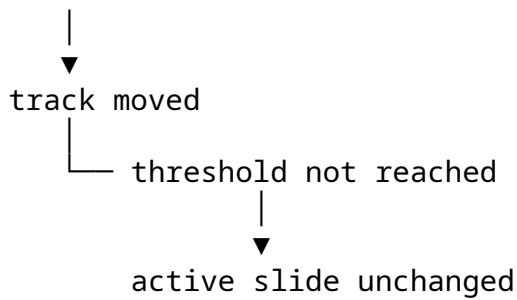
Сначала архитектура отвечает на проблему; название паттерна появляется потом.

13. Значение slidemove и slidechange

Один из самых ценных уроков реализации скрыт в различии между физическим движением и семантическим изменением.

Track может двигаться, не меняя активный slide.

небольшой drag



Поэтому приложению нужны два разных понятия:

`slidemove`

= движение началось / происходит

`slidechange`

= активный `slide` действительно изменился

Это не просто деталь реализации. Это защищает синхронизацию между Slider, аудиоальбомом и ссылкой Shop.

Несколько пикселей drag не должны переключать альбом.

Это хороший пример того, как архитектурная ясность помогает обнаружить и исправить тонкий поведенческий баг.

14. Почему autoscroll - больше, чем timer

На первый взгляд autoscroll звучит просто:

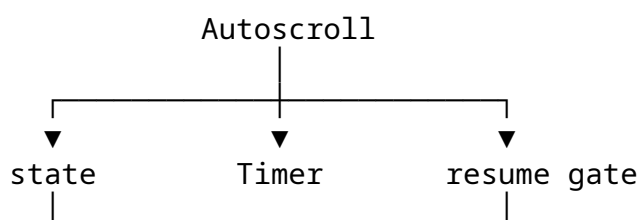
`setInterval(next)`

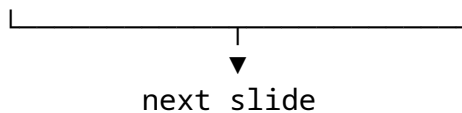
Но реальная задача взаимодействия сложнее.

На autoscroll могут влиять:

manual interaction
hover/focus pauses
dragging
tab visibility
audio playback
explicit enable/disable
transitions
время возобновления

Поэтому в проекте autoscroll рассматривается как собственная подсистема.





Поэтому имеет смысл LOCKED и имеет смысл `_tryResumeAutoscroll()`: возобновление - это решение, а не просто вызов метода.

15. Почему LOCKED отличается от OFF

Различие здесь семантическое.

OFF

= возможность намеренно отключена

LOCKED

= возможность существует, но временно запрещена другим условием

Наиболее наглядный пример - воспроизведение альбома.

album starts



lock autoscroll



play album without competing motion

Когда альбом ставится на паузу или заканчивается, приложение может решить, можно ли снова запускать autoscroll.

Обычный boolean размывал бы эти два смысла.

State model оставляет их различными.

16. Почему у MediaPlayer собственная модель состояния

У native `<audio>` уже есть собственное состояние, но оно не знает семантический контекст приложения.

Такое свойство, как `audio.paused`, не может ответить на все вопросы уровня приложения.

Например:

paused main theme

и:

paused album

для браузера оба означают "paused", но для приложения это разные ситуации.

Поэтому MediaPlayer вводит собственную семантическую модель режимов.

```
native media state
      +
application playback context
      v
AudioPlayer FSM
```

Это ещё один пример общего принципа проекта:

Состояние браузера/фреймворка не обязательно совпадает с состоянием домена.

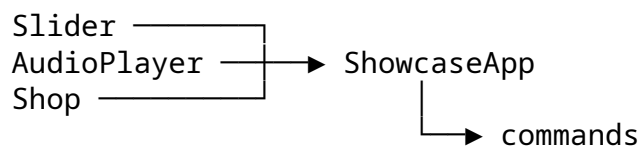
17. Почему компоненты общаются через события

Архитектура намеренно не строит сеть прямых зависимостей.

Плохой вариант:



Предпочтительный вариант:



Компонент сообщает:

"произошло значимое событие"

А приложение решает:

"что должно произойти в других частях из-за этого"

Это практическая причина событийной архитектуры проекта.

18. Проект использует паттерны как язык, а не как украшение

В архитектуре можно узнать несколько паттернов, но проект не относится к ним как к коллекции трофеев.

Mediator-like
-> ShowcaseApp

Observer / Pub-Sub-like
-> EventManager + custom events

Chain of Responsibility-like
-> _pipe result protocol

Strategy-like
-> configurable managers

Template Method-like
-> inheritance hooks in Slider

Command-like
-> Button

FSM
-> local state models

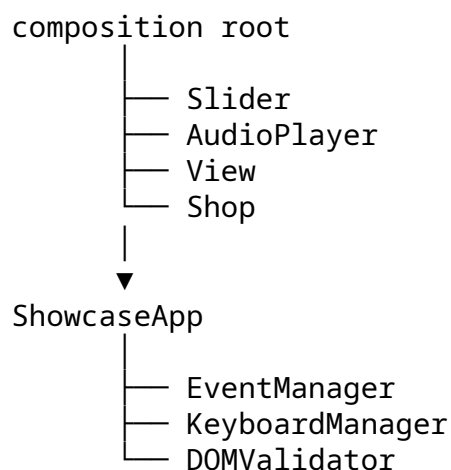
Часть идей правильнее описывать как принципы:

Dependency Injection
Inversion of Control
Composition
Fail-fast validation
Component encapsulation
Configuration-driven behavior

Такое различие важно, потому что архитектурный словарь должен быть честным.

19. Почему важны DI и composition

Зависимости собираются снаружи, а не ищутся глобально.



Так object construction и application policy остаются рядом с composition boundary.

И систему легче читать: глядя на composition root, разработчик видит, из чего состоит приложение.

20. Почему проект избегает god object

Самым простым способом построить такое приложение было бы поместить всё в ShowcaseApp. Тогда он мог бы управлять:

```
DOM
slider indices
transitions
dragging
autoscroll
audio
track selection
shop links
keyboard shortcuts
```

Это работало бы.

Но получился бы объект, который знает слишком много.

Вместо этого:

```
ShowcaseApp
  -> координирует
```

```
Slider
  -> владеет механикой Slider
```

```
AudioPlayer
  -> владеет механикой воспроизведения
```

```
Managers
  -> владеют переиспользуемыми механизмами
```

Поэтому coordinator может быть широким по области ответственности, но неглубоким по знанию внутренней логики доменных компонентов.

21. Почему validation - часть философии

Проект предпочитает явные контракты молчаливым догадкам.

Если компонент требует элемент, он должен существовать.

Если configuration ссылается на handler, handler должен существовать.

Если state token неверен, setter должен его отвергнуть.

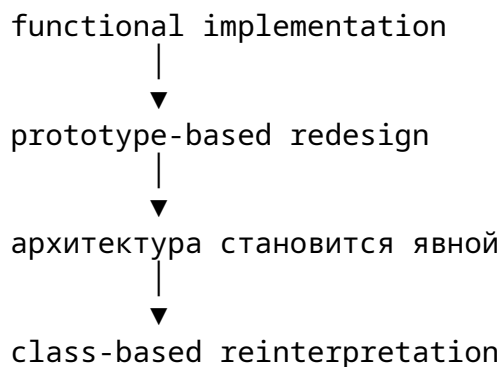
```
assumption
  |
  ▼
validate at boundary
  |
  └─ valid -> continue
  └─ invalid -> fail early
```

Это не только техника реализации, но и дизайнерская философия.

Скрытый fallback может сделать архитектурную ошибку менее заметной. Fail-fast контракт показывает её там, где она была создана.

22. Проект как этап развития

Архитектура также отражает личный переход к тому, как рассматривается одна и та же идея.



Важен не синтаксис.

Важно растущее умение рассуждать о:

ответственности
границах
контрактах
состоянии
композиции
полиморфизме
переиспользовании

Поэтому вторая реализация служит точкой сравнения: архитектура выдерживает переход от одного объектно-ориентированного синтаксиса к другому.

23. Что в итоге действительно достигнуто

Получившийся результат - это не просто "Slider с набором функций".

Он показывает, что исходную идею можно разложить на:

независимые доменные компоненты
+
переиспользуемую инфраструктуру
+
локальные конечные автоматы
+
полиморфные уровни возможностей
+

декларативную настройку взаимодействия
+
координацию на уровне приложения

Архитектура оказалась полезной и во время debugging и развития проекта.

Тонкие проблемы стало проще локализовать, потому что ответственности были видимы. Ошибку в семантике `slidechange` можно было искать на границе между движением и фиксацией. Поведение `autoscroll` можно было рассуждать через отдельное состояние и условия возобновления. `Keyboard propagation` можно было рассматривать как `pipeline`, а не как набор несвязанных обработчиков.

В этом и состоит практический результат архитектурной работы: архитектура не только документирует готовую систему. Она помогает находить и объяснять поведение во время разработки.

24. Почему точное описание здесь - "сложно, но контролируемо"

Проект намеренно сложнее, чем необходимо для его визуальной задачи.

Но сложность организована.

complexity

- inheritance -> структура возможностей
- managers -> повторное использование механизмов
- FSM -> ясность состояния
- events -> слабая связанность
- pipe -> кооперация при вводе
- validation -> явные контракты

Архитектура работает тогда, когда новую возможность можно мысленно куда-то поместить.

new concern



Where does it belong?

- domain behavior -> component
- reusable mechanism -> manager
- stateful behavior -> owning FSM
- cross-component rule -> ShowcaseApp

Это и есть настоящий критерий контролируемости.

25. Почему эта архитектура интересна не только в рамках этого проекта

Конкретный UI небольшой, но архитектурные вопросы - нет.

В большом приложении всё равно придётся решать:

кто владеет состоянием?
кто владеет инфраструктурой?
кто координирует компоненты?
как распространяются события?
что следует сделать переиспользуемым?
где нужен полиморфизм?
когда абстракция оправдана?
когда паттерн не нужен?

Поэтому проект выступает компактной моделью задач, возникающих и в системах большего масштаба.

Преимущество небольшой и понятной системы в том, что здесь ещё можно видеть причинно-следственную связь.

26. Финальный взгляд

Самое важное достижение проекта - не количество классов, менеджеров или паттернов. Это изменение способа мышления:

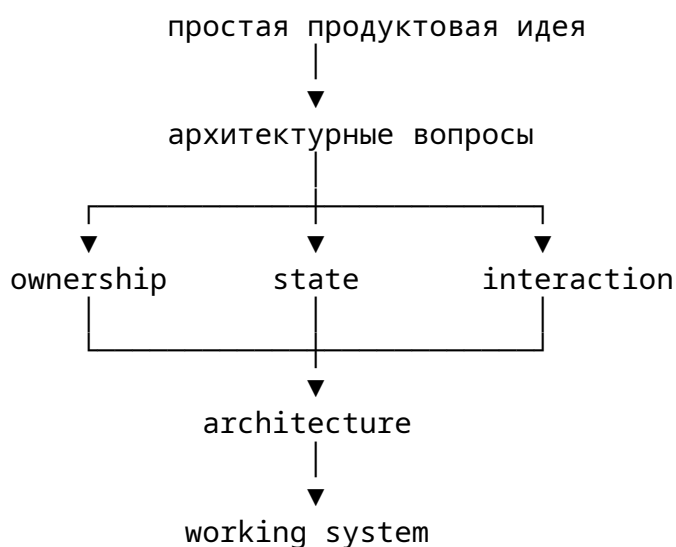
"Как заставить эту возможность работать?"

сменяется на:

"Что стоит за этой возможностью как ответственность?
Кто должен ею владеть?
Что должен знать компонент?
Какой механизм можно переиспользовать?
Какое состояние здесь действительно моделируется?
Как остальные части системы должны узнать, что произошло?"

Именно поэтому проект оказался больше первоначальной UI-идеи.

Приложение - видимый результат. Архитектурное мышление - настоящий предмет работы.



И философию проекта можно выразить одной фразой:

Строй столько архитектуры, сколько нужно, чтобы система оставалась понятной, расширяемой и контролируемой, - но только там, где реальная проблема действительно даёт этой архитектуре причину существовать.

Это не production-framework, замаскированный под Slider. Это сознательная лаборатория для изучения того, как проектировать интерактивную систему из ответственности, состояния, контрактов и взаимодействия, а не из одних только функций.